



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich



Vector Search with Out-of-Distribution Queries

Semester Project Report

Anil Eren Göçer

`agoecer@ethz.ch`

Systems Group
Department of Computer Science
ETH Zürich

Supervisors:

Prof. Dr. Yannis Chronis
Vasileios Mageirakos

July 15, 2026

Abstract

Vector search indexes are typically built from data vectors alone, implicitly assuming that queries follow the same distribution. In multimodal retrieval and sparse attention, this assumption breaks down, producing out-of-distribution (OOD) queries that are harder to process and can substantially degrade recall–latency performance on inverted file (IVF) indexes.

We study this effect across eleven benchmark datasets, characterizing OOD behavior as a continuous spectrum via Mahalanobis distance. We show that performance degradation is driven by increased *scatteredness* of nearest neighbors across partitions and lower *Partition Probe Accuracy* (PPA) of centroid-based probing.

To address this, we propose `IVFFlat_Q` and `QueryWeighted_IVFFlat`, two IVF construction strategies that incorporate sample queries. The `QueryWeighted_IVFFlat` index consistently outperforms standard `IVFFlat` on OOD datasets, indicating that robust indexes should leverage information from both query and data distributions.

Contents

Abstract	i
1 Introduction	1
1.1 Vector Search Problem	1
1.2 Vector Search Indexes	2
1.2.1 Graph-based Indexes	2
1.2.2 Partitioning-based (IVF) Indexes	2
2 Out-of-Distribution Queries	4
2.1 Mismatch in the Distributions of Query and Data Vectors	4
2.2 Characterization of Out-of-Distribution Queries	4
2.2.1 Geometric Analysis of Data and Query Vectors	4
2.2.2 Quantifying OOD Behavior	7
2.3 Performance Comparison: ID vs. OOD Queries	9
3 Analyzing Performance Degradation under OOD Queries	12
3.1 Partition-wise Distribution of Query Nearest Neighbors	12
3.2 Partition Probe Accuracy	15
3.3 Distribution of Partition Probe Frequencies	17
4 Towards Out-of-Distribution Robust IVF Index Design	19
4.1 The Design Spectrum of Vector Search Indexes	19
4.1.1 Query-Driven IVF Index Construction: IVFFlat_Q	19
4.1.2 A Hybrid IVF Index Construction Strategy: QueryWeighted_IVFFlat	20
4.2 Impact of Query Distribution Information Injection into Index Construction on Performance	20
4.3 Understanding the Performance Gains of Query-Aware Index Construction	22

Introduction

1.1 Vector Search Problem

Recent advances in deep learning have enabled the representation of unstructured data, such as text, images, and audio, as numerical vectors known as embeddings [3, 9]. These embedding vectors typically reside in high-dimensional spaces, often ranging from 128 to 3072 dimensions, and are designed to capture the semantic properties of the underlying data. Consequently, objects that are semantically similar are mapped to nearby points in the embedding space, allowing geometric proximity between vectors to serve as a measure of semantic similarity.

This representation naturally motivates the need for efficient algorithms and systems that can retrieve the k most similar vectors to a given query vector q , in order to identify the most relevant data instances. This task is commonly referred to as Nearest Neighbor Search or Vector Search in the literature. More formally, given a dataset of vectors X , where $\forall x \in X, x \in \mathbb{R}^d$, and a query vector $q \in \mathbb{R}^d$, the objective is to retrieve a subset $Y \subseteq X$ such that $|Y| = k$, where for each $y \in Y$, y is among the k vectors in X that are closest to q under a chosen distance metric (e.g., Euclidean distance or cosine distance).

The naive solution to this problem is straightforward: compute the distance between the query vector q and every vector in the dataset X , and then perform a brute-force sort to identify the closest matches. However, this exhaustive approach is computationally expensive, particularly for large-scale datasets, as it has a time complexity of $O(N \cdot d)$, where N is the number of vectors and d is the dimensionality of the vectors. Consequently, most practical systems rely on approximate methods, reformulating the task as Approximate Nearest Neighbor Search to achieve significantly better efficiency with only a small loss in accuracy.

To evaluate the accuracy of vector search systems and algorithms, a commonly used metric is recall. In particular, $\text{recall}@k$ measures how well an approximate method retrieves the true nearest neighbors. Formally, let Y_{true} denote the exact set of k nearest neighbors to the query q , and let Y_{approx} denote the set of k neighbors returned by an approximate algorithm. Then, $\text{recall}@k$ is defined as:

$$\text{Recall}@k = \frac{|Y_{\text{true}} \cap Y_{\text{approx}}|}{k}.$$

This metric captures the fraction of true nearest neighbors that are successfully retrieved by the approximate method.

In addition to accuracy, the performance of vector search systems is typically evaluated in terms of efficiency. The most common efficiency metrics are latency and throughput.

Latency measures the time required to process a single query, while throughput is typically measured as the number of queries processed per second (queries per second, QPS).

1.2 Vector Search Indexes

Similar to traditional database systems, vector databases employ specialized indexing structures to accelerate vector search while preserving high recall (typically $\geq 90\%$). Although a wide range of vector search indexes has been proposed in recent years, they can broadly be categorized into two main classes: graph-based indexes and partitioning-based (IVF) indexes.

1.2.1 Graph-based Indexes

Graph-based indexes represent vector search structures as a graph, where each node corresponds to a data vector and edges connect similar vectors. The intuition is that nearest neighbor search can be formulated as a graph traversal problem: starting from an entry node, the algorithm greedily explores neighboring nodes that are closer to the query, progressively moving toward regions of higher similarity.

Hierarchical Graph Structures

Hierarchical graph-based indexes organize vectors into multiple layers of graphs with different densities. Higher layers contain fewer nodes and provide long-range navigation, while lower layers contain more nodes for fine-grained search. During query processing, the search begins at the top layer and progressively descends to lower layers, refining the search at each level.

A prominent example of this approach is HNSW (Hierarchical Navigable Small World graphs) [7], which builds a multi-layer graph where each layer is a proximity graph. The hierarchical structure allows logarithmic-time navigation in practice and is widely used due to its strong balance between accuracy and efficiency.

Flat Graph Structures

Flat graph-based indexes use a single-layer proximity graph where all vectors reside in the same graph structure. Query processing starts from one or more entry points and iteratively traverses the graph by moving to neighboring nodes that improve proximity to the query vector.

Vamana [10] is a representative example of a flat graph index. It constructs a sparse yet well-connected graph designed to ensure efficient greedy navigation. Unlike hierarchical methods, flat graphs rely entirely on local connectivity and graph quality to guarantee search performance, making graph construction a critical component of the system.

1.2.2 Partitioning-based (IVF) Indexes

Partitioning-based indexes, commonly referred to as Inverted File (IVF) indexes, accelerate vector search by decomposing the global search space into a set of smaller subspaces. The

central idea is to avoid scanning the entire dataset by restricting the search to a subset of candidate vectors that are most likely to contain the nearest neighbors of the query. This is achieved through a two-stage process: (i) clustering (space partitioning) and (ii) refined search within selected partitions.

Clustering

Given a dataset $X \subset \mathbb{R}^d$, IVF indexes first learn a set of K cluster centroids:

$$C = \{c_1, c_2, \dots, c_K\}, \quad c_i \in \mathbb{R}^d$$

typically obtained using a clustering algorithm such as k-means. Each vector $x \in X$ is then assigned to its nearest centroid:

$$\text{assign}(x) = \arg \min_{c_i \in C} \|x - c_i\|$$

This process partitions the dataset into K disjoint inverted lists (also called posting lists), where each list corresponds to a centroid and contains all vectors assigned to it.

Query Processing in IVF

At query time, a query vector $q \in \mathbb{R}^d$ is not compared against all vectors in X . Instead, the system first identifies the n_{probe} nearest centroids to q :

$$C_q = \text{Top-}n_{\text{probe}}(c_i \in C, \|q - c_i\|)$$

where n_{probe} is a critical search parameter controlling the recall-latency trade-off.

Once the relevant centroids are selected, the search is restricted to the union of their associated inverted lists:

$$X_q = \bigcup_{c_i \in C_q} \text{list}(c_i)$$

A brute-force distance computation is then performed only over X_q , and the top- k nearest neighbors are returned.

Out-of-Distribution Queries

2.1 Mismatch in the Distributions of Query and Data Vectors

Traditional vector search indexes implicitly assume that both the indexed data vectors and the query vectors are drawn from the same underlying distribution. Consequently, index construction is performed solely based on the data vectors, without considering potential differences in the query distribution.

However, in many modern applications that rely on vector search, query vectors often follow a distribution that differs from that of the indexed data vectors. One common example is multimodal retrieval, where queries and indexed items belong to different modalities; for instance, query embeddings may be generated from text, while the indexed data vectors are derived from images. Similarly, in sparse attention mechanisms that employ KNN search within large language models, query (Q) vectors and key (K) vectors (i.e., the indexed data vectors) originate from different distributions [4]. These scenarios violate the assumption of distributional similarity between queries and data vectors.

As noted in [1], this distributional mismatch increases the computational effort required to process a query and achieve a given recall target, a property referred to as query hardness.

2.2 Characterization of Out-of-Distribution Queries

In this section, we aim to characterize what makes out-of-distribution (OOD) queries distinct from in-distribution (ID) queries. More specifically, we analyze the properties of the datasets presented in Table 2.1.

2.2.1 Geometric Analysis of Data and Query Vectors

To gain insight into the geometric structure of the datasets, we project both data and query vectors onto a two-dimensional space using Principal Component Analysis (PCA) and visualize the resulting distributions.

As shown in Figure 2.1, the query and data distributions exhibit a near-perfect overlap for the SIFT and Deep datasets. Although the query vectors for the GIST dataset appear concentrated near the center of the data distribution, this is primarily due to the relatively small number of query vectors compared to data vectors. Nevertheless, the queries remain in-distribution, as their distribution is a subset of the data distribution in the embedding

space. This overlap arises because both data and query vectors are sampled from the same underlying distribution, which is why these queries are classified as in-distribution (ID).

Dataset	Queries	Domain
SIFT [6]	ID	Unimodal retrieval
DEEP [12]	ID	Unimodal retrieval
GIST [8]	ID	Unimodal retrieval
MS COCO [4]	OOD	Multi-modal retrieval
ImageNet [4]	OOD	Multi-modal retrieval
LAION [4]	OOD	Multi-modal retrieval
Yandex-T2I [4]	OOD	Multi-modal retrieval
WebVid [4]	OOD	Multi-modal retrieval
Yi [4]	OOD	Sparse attention
LLaMA [4]	OOD	Sparse attention
PubMed23 [11]	OOD	Unimodal retrieval

Table 2.1: Datasets

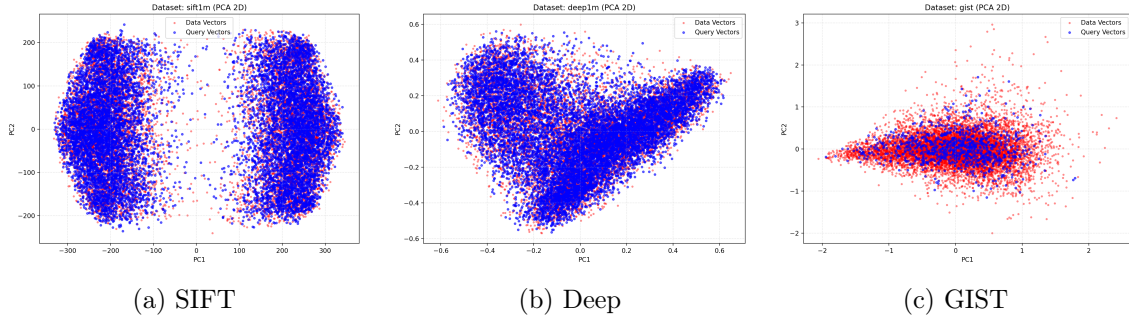


Figure 2.1: Two-dimensional PCA projections of data and query vector distributions for datasets where queries are in-distribution (ID) with respect to the data. Red points represent data vectors, while blue points represent query vectors.

In contrast, a clear separation between query and data distributions can be observed for the MS COCO, ImageNet, LAION, and WebVid datasets, as shown in Figure ???. This indicates that the query and data vectors originate from different underlying distributions. In these cases, the distributional gap is primarily driven by modality differences between queries and data vectors; for example, queries are represented as text embeddings, whereas data vectors are image embeddings.

Although the Yandex T2I and PubMed23 datasets are labeled as having OOD queries in [4, 11], respectively, the corresponding distributional gap is not clearly pronounced. We later quantify this property using the Mahalanobis distance, which suggests that OOD behavior is not a binary characteristic but rather lies on a continuous spectrum.

Similarly, a clear distribution gap between data and query vectors can be observed for the Yi and LLaMA datasets in Figure ??. This arises from the use of separate projection matrices, W_Q and W_K , for Query (Q) and Key (K) vectors in the attention mechanism. As a result, although they interact within the same embedding space, they are mapped into different regions, leading to distinct distributions.

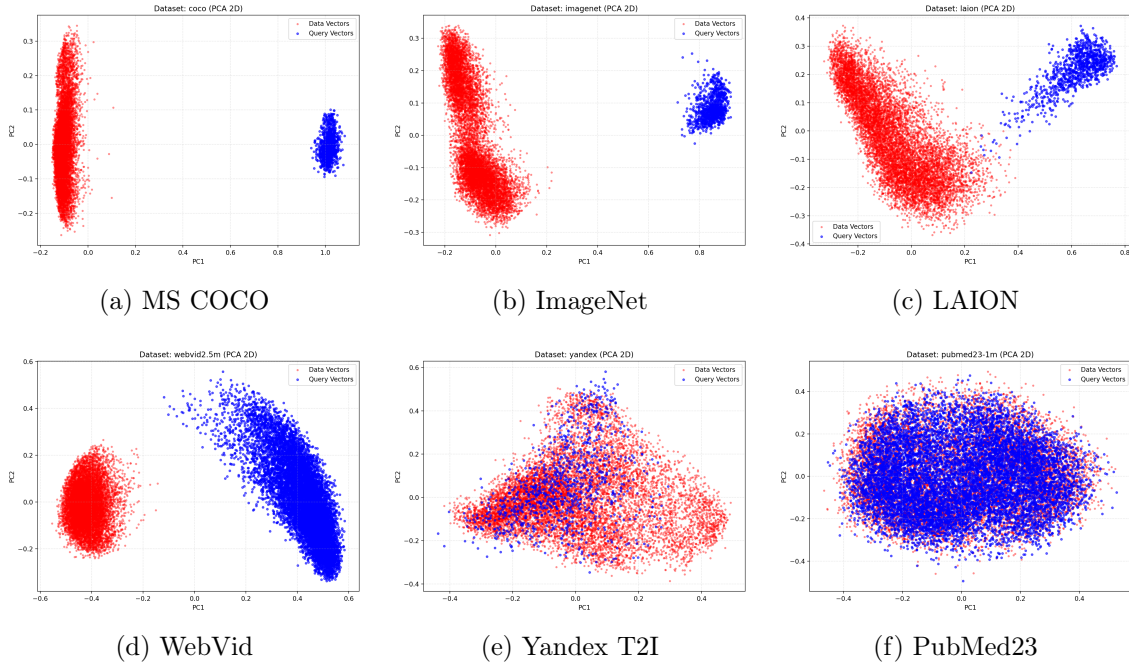


Figure 2.2: Two-dimensional PCA projections of data and query vector distributions for multi-modal/unimodal retrieval datasets in which queries are OOD relative to the data. Red points represent data vectors, blue points represent query vectors.

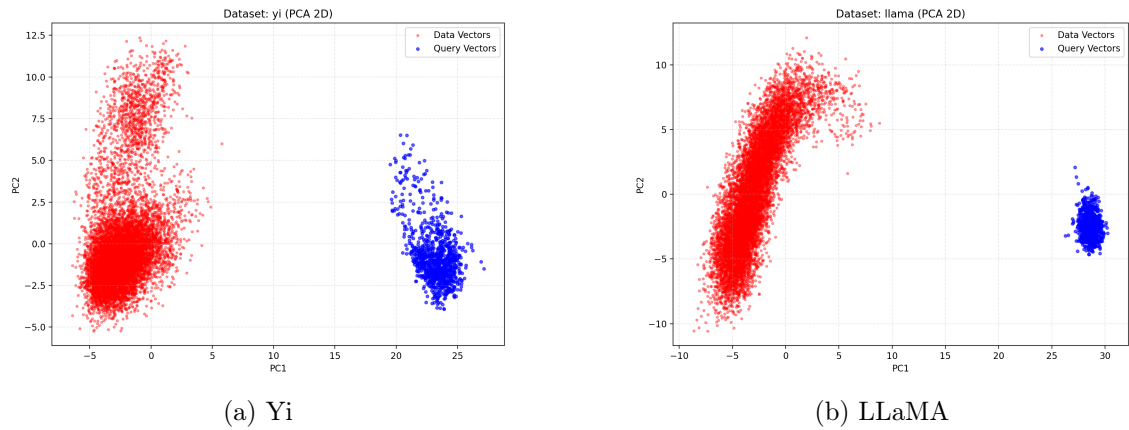


Figure 2.3: Two-dimensional PCA projections of data and query vector distributions for sparse attention datasets in which queries are OOD relative to the data. Red points represent data vectors, blue points represent query vectors.

Dataset	Ratio of Mean Mahalanobis Distances (Query / Data)
SIFT	0.9998
Deep	0.9979
GIST	0.9882
MS COCO	17.4172
ImageNet	13.9482
LAION	7.5256
WebVid	11.0639
Yandex T2I	1.5798
PubMed23	1.1275
Yi	7.5520
LLaMA	13.3062

Table 2.2: Ratio of mean Mahalanobis distances between query and data vectors across datasets.

2.2.2 Quantifying OOD Behavior

To quantitatively evaluate the degree of out-of-distribution (OOD) behavior in each dataset, we compute and visualize the Mahalanobis distance of both data vectors and query vectors with respect to the data distribution.

Mahalanobis distance is a metric that measures the distance between a point and a distribution while accounting for the covariance structure of the data. Unlike Euclidean distance, it is invariant to scaling and correlations between dimensions, making it particularly suitable for assessing distances in high-dimensional embedding spaces. Formally, given a point $x \in \mathbb{R}^d$, a distribution with mean vector $\mu \in \mathbb{R}^d$, and covariance matrix $\Sigma \in \mathbb{R}^{d \times d}$, the Mahalanobis distance is defined as:

$$D_M(x) = \sqrt{(x - \mu)^\top \Sigma^{-1} (x - \mu)}.$$

In this work, we use this metric to measure how far query and data vectors deviate from the empirical data distribution, where μ and Σ are estimated from the data vectors of each dataset.

As shown in Figure 2.4, the Mahalanobis distance distributions of query and data vectors in the SIFT, Deep, and GIST datasets exhibit near-perfect overlap, confirming that the queries are in-distribution with respect to the data distribution. This is further supported by the ratio of mean distances being close to 1, indicating a low degree of out-of-distribution (OOD) behavior, as reported in Table 2.2.

In contrast, for the MS COCO, ImageNet, LAION, and WebVid datasets, query vectors are substantially farther from the data distribution compared to data vectors, as shown in Figure 2.5. This observation is consistent with the geometric structure discussed earlier. The ratio of mean distances ranges from 7.5256 to 17.4173, significantly exceeding 1 and indicating a much stronger degree of out-of-distribution (OOD) behavior.

For the Yandex-T2I and PubMed23 datasets, we also observe a distributional gap between the distances of query and data vectors, although it is less pronounced compared

to the other datasets. This is further reflected in the mean distance ratios of 1.5798 and 1.1275, respectively, which are only slightly greater than 1. This indicates that the queries in these datasets exhibit some degree of out-of-distribution (OOD) behavior, but to a much lesser extent than in the other cases.

Similarly, a comparable gap in Mahalanobis distances is observed for the Yi and LLaMA datasets, with mean distance ratios of 7.5520 and 13.3062, respectively, which align with the geometric patterns discussed earlier.

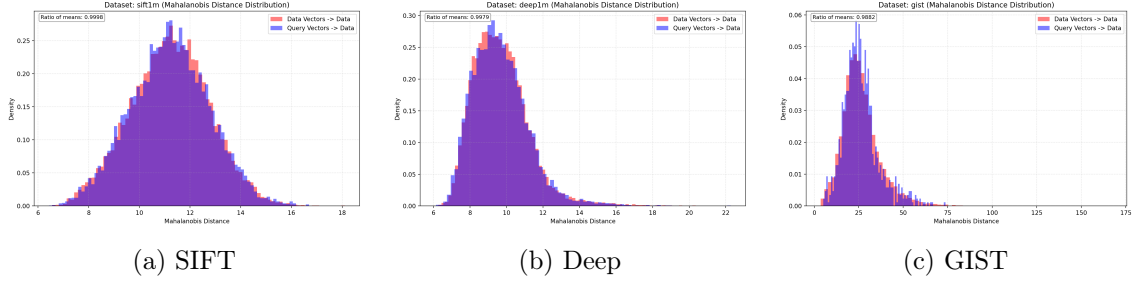


Figure 2.4: Distributions of Mahalanobis distances from query vectors to the data distribution and from data vectors to the data distribution for datasets where queries are in-distribution (ID) with respect to the data. Red bars represent data vectors, while blue bars represent query vectors. The vertical axis shows density, and the horizontal axis shows Mahalanobis distance.

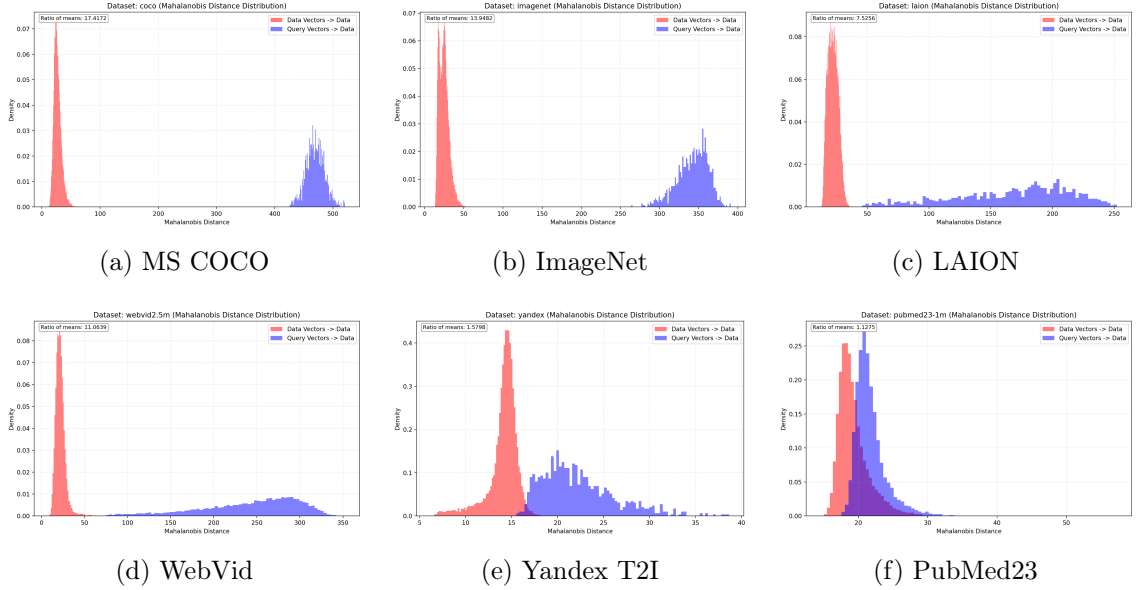


Figure 2.5: Distributions of Mahalanobis distances from query vectors to the data distribution and from data vectors to the data distribution for multi-modal/unimodal retrieval datasets where queries are out-of-distribution (OOD) with respect to the data. Red bars represent data vectors, while blue bars represent query vectors. The vertical axis shows density, and the horizontal axis shows Mahalanobis distance.

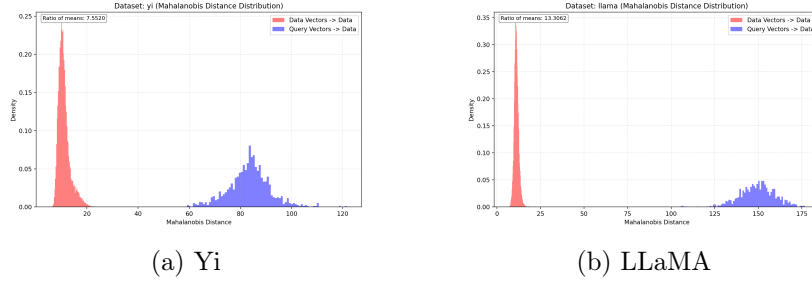


Figure 2.6: Distributions of Mahalanobis distances from query vectors to the data distribution and from data vectors to the data distribution for sparse attention datasets where queries are out-of-distribution (OOD) with respect to the data. Red bars represent data vectors, while blue bars represent query vectors. The vertical axis shows density, and the horizontal axis shows Mahalanobis distance.

2.3 Performance Comparison: ID vs. OOD Queries

In this section, we investigate the impact of out-of-distribution (OOD) queries on vector search performance. For each dataset, we construct an IVFFlat index using FAISS with 1000 partitions over the data vectors. We then evaluate recall and latency by gradually increasing the value of n_{probe} under two experimental settings. In the first setting, randomly sampled data vectors are used as queries against the index, representing the in-distribution (ID) case. In the second setting, the original query vectors provided with each dataset are used to query the same index, representing the out-of-distribution (OOD) case.

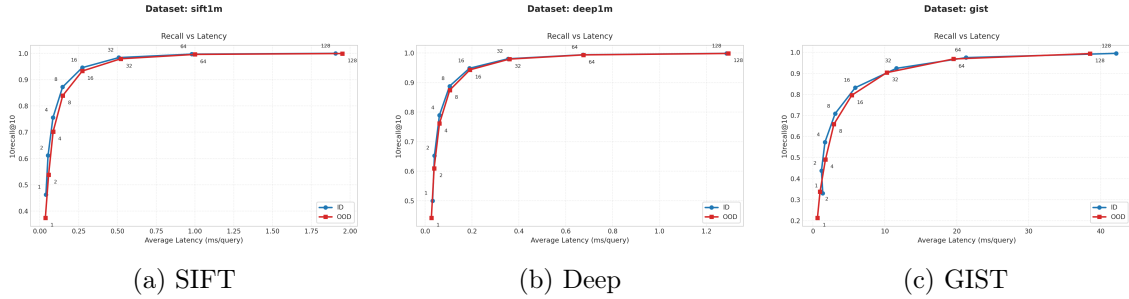


Figure 2.7: Recall–latency curves for datasets exhibiting only a negligible (almost zero) degree of OOD behavior. The red curve corresponds to the case where the original query vectors provided with the dataset are used (labeled as OOD), while the blue curve corresponds to the case where randomly sampled data vectors are used as queries (labeled as ID). The horizontal axis represents Recall@10, and the vertical axis shows query latency in milliseconds (ms).

As shown in Figure 2.7, there is almost no difference between the recall–latency curves of the ID setting (data vectors used as queries) and the OOD setting (original query vectors). This indicates that the SIFT, Deep, and GIST datasets exhibit an almost negligible degree of OOD behavior and are commonly regarded as in-distribution (ID) datasets in the literature. These findings are consistent with the Mahalanobis distance results reported in Table 2.2, where the mean distance ratios are approximately 1 for all three datasets.

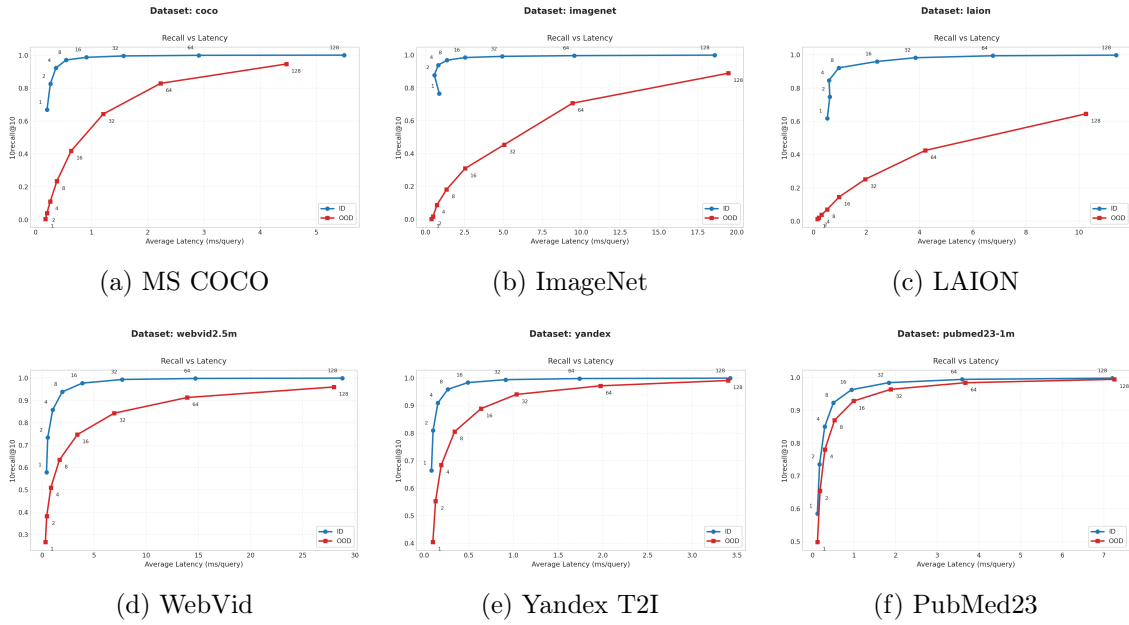


Figure 2.8: Recall-latency curves for multi-modal/unimodal retrieval datasets with varying non-negligible degree of OOD behavior. The red curve corresponds to OOD queries, while the blue curve corresponds to ID queries. The horizontal axis is Recall@10 and the vertical axis is latency (ms).

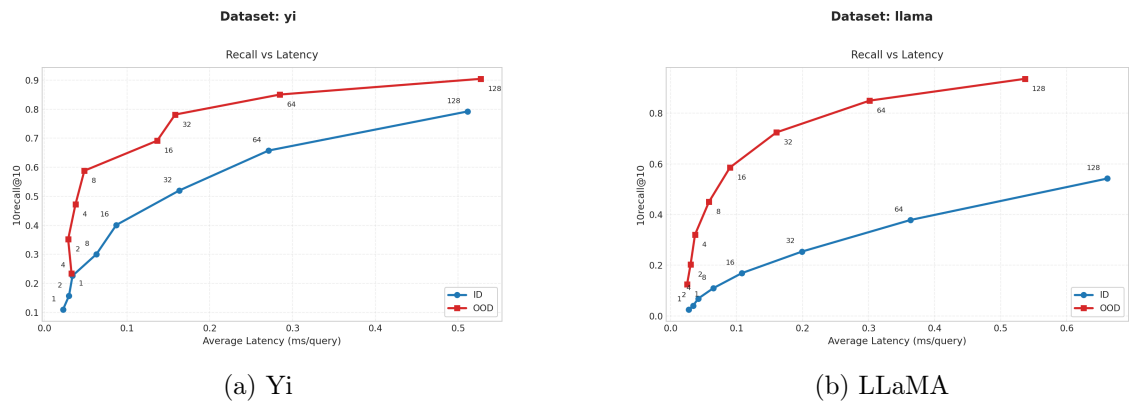


Figure 2.9: Recall-latency curves for sparse attention retrieval datasets. The red curve corresponds to OOD queries, while the blue curve corresponds to ID queries. The horizontal axis is Recall@10 and the vertical axis is latency (ms).

On the other hand, a noticeable performance gap is observed for the multi-modal and unimodal retrieval datasets, as shown in Figure 2.8. For instance, reaching 80% Recall@10 is nearly $20\times$ slower for OOD queries in the ImageNet dataset. The gap is substantial for MS COCO, ImageNet, LAION, and WebVid, while it is smaller for Yandex T2I and PubMed23, though still present.

These observations are consistent with the results in Table 2.2. The ratio of mean distances (query/data) for MS COCO, ImageNet, LAION, and WebVid ranges from 7.53 to 17.42, significantly exceeding 1, whereas Yandex T2I and PubMed23 have ratios of 1.58 and 1.12, respectively. This explains the pronounced performance gap in the first group of datasets and the milder gap in the second group. The relatively small but still noticeable performance gap for PubMed23 is likely due to the unimodal nature of both queries and data vectors.

Due to the observed performance degradation when using query vectors compared to using data vectors as queries, these datasets are commonly referred to as out-of-distribution (OOD) datasets in the literature.

For the sparse attention datasets (Yi and LLaMA), we still observe a large performance gap, as seen in Figure 2.9. However, in this case OOD queries are faster, which contradicts prior findings in the literature suggesting that OOD queries are typically slower. The results in Table 2.2 do not fully explain this behavior, as the corresponding ratios are 7.55 and 13.30, respectively, both well above 1. While we provide some discussion in the following chapter, the underlying reason for the faster OOD queries in sparse attention datasets remains unclear and requires further investigation.

Analyzing Performance Degradation under OOD Queries

In the previous chapter, we observed that out-of-distribution (OOD) queries generally lead to performance degradation, with the exception of sparse attention datasets. In this chapter, we aim to understand the underlying reasons for this degradation by examining key insights from the constructed index itself and the query processing perspective.

3.1 Partition-wise Distribution of Query Nearest Neighbors

We observe that as the degree of out-of-distribution (OOD) behavior increases, the nearest neighbors of a query become distributed across more partitions. To quantify this phenomenon, we introduce the notion of *scatteredness*.

Definition (Scatteredness). Let q be a query vector, $N_k(q)$ its set of k nearest neighbors, and $P(x)$ the partition containing data vector x . The *scatteredness* of q is defined as the number of distinct partitions containing its k nearest neighbors:

$$\text{Scatteredness}(q, k) = |\{P(x) \mid x \in N_k(q)\}|.$$

Scatteredness therefore represents a lower bound on the number of partitions that must be probed to retrieve all k nearest neighbors. Its value ranges from 1, when all neighbors lie in the same partition, to k , when each neighbor belongs to a different partition.

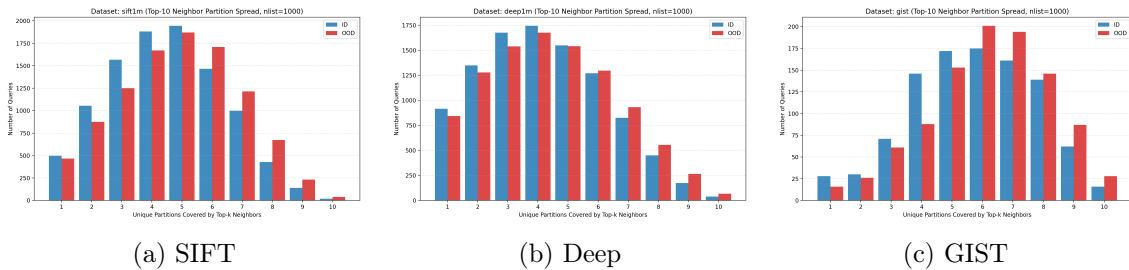


Figure 3.1: Distribution of neighbor scatteredness for $k = 10$ in the SIFT, Deep, and GIST datasets. Blue bars correspond to the ID setting (data vectors used as queries), while red bars correspond to the OOD setting (the original query vectors provided with the datasets). The vertical axis shows the number of queries, and the horizontal axis represents the neighbor scatteredness, ranging from 1 to $k = 10$.

For the datasets classified as in-distribution (ID), there is almost no difference between the scatteredness distributions of the ID setting, where data vectors are used as queries, and the OOD setting, where the original query vectors are used. This observation is consistent with our hypothesis that scatteredness is positively correlated with the degree of out-of-distribution (OOD) behavior, as quantified by the Mahalanobis distance. As shown in Figure 3.1, the distributions are nearly identical, with only a negligible difference for scatteredness values up to 3 across all three datasets.

When examining the scatteredness distributions for ID and OOD queries in the multimodal retrieval datasets, as shown in Figure 3.2, we observe a substantial gap between the two settings. For instance, in the MS COCO dataset, approximately 500 out of 1000 ID queries have scatteredness equal to 2, meaning that all their $k = 10$ nearest neighbors are contained within only two partitions. In contrast, this is the case for only about 100 out of 1000 OOD queries. A closer look at the distributions further reveals that the mean scatteredness for ID queries is close to 2, whereas it increases to around 5 for OOD queries. Similar patterns are observed across other datasets in this category, and the magnitude of this gap aligns with the Mahalanobis distance mean ratios reported in Table 2.2.

On the other hand, we do not observe a significant distributional gap between the ID and OOD settings for the sparse attention datasets (Yi and LLaMA), which are also classified as OOD. As shown in Figure 3.3, the scatteredness under the OOD setting for the Yi dataset is even lower than that of the ID setting. These observations are consistent with our earlier results on performance (recall-latency curves), although they do not provide an explanation for the underlying behavior.

The higher the scatteredness of a query, the more effort is required to achieve a given target recall. This is because low scatteredness implies that the nearest neighbors can be found by probing only a small number of partitions, which in turn leads to lower latency and improved performance. This effect becomes more pronounced as k increases, since k serves as an upper bound on scatteredness.

Moreover, higher scatteredness also leads to increased entropy among the nearest neighbors, which in turn degrades the effectiveness of centroid-proximity-based probing, i.e., prioritizing partitions whose centroids are closest to the query. As a result, more partitions need to be probed, since many of the initially selected partitions do not contain any of the k nearest neighbors (i.e., false positives). This phenomenon is discussed in detail in the next section.

In summary, scatteredness provides a unified measure of how nearest neighbors are distributed across partitions and how this structure changes under out-of-distribution (OOD) conditions. For in-distribution (ID) datasets, scatteredness is nearly identical between ID and OOD query settings, consistent with their low Mahalanobis distance ratios. In contrast, multimodal retrieval datasets show a clear increase in scatteredness under OOD queries, aligning with higher Mahalanobis distances, while sparse attention datasets do not follow this trend. Overall, higher scatteredness leads to increased partition probing and degraded recall-latency performance, establishing it as a key factor linking OOD behavior to efficiency loss in IVF-based vector search.

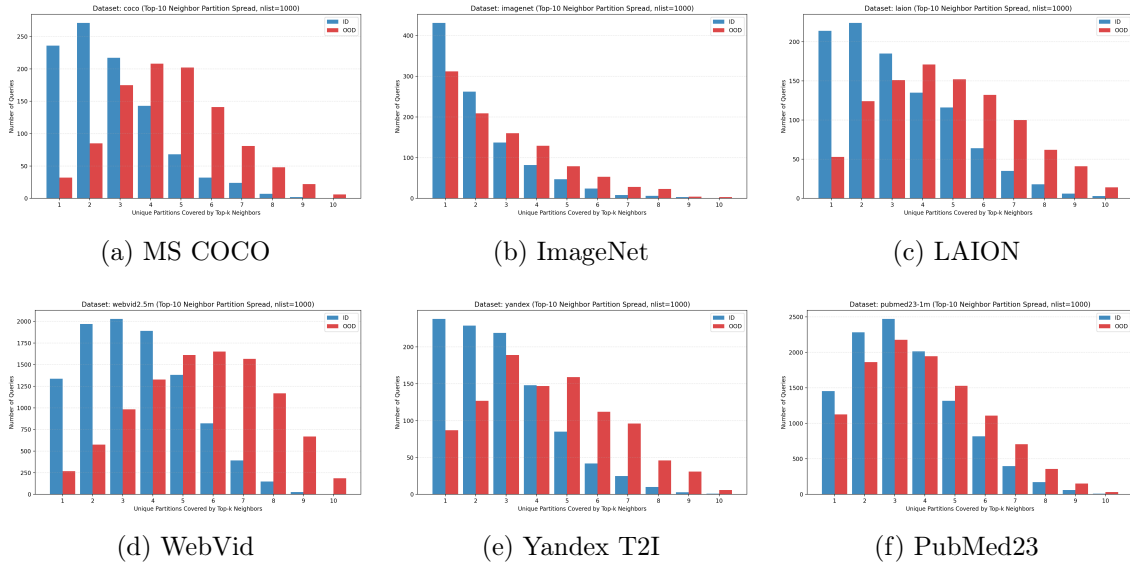


Figure 3.2: Distribution of neighbor scatteredness for $k = 10$ in multi-modal retrieval datasets. Blue bars correspond to ID queries, red bars to OOD queries. The x-axis shows neighbor scatteredness (1 to $k = 10$), and the y-axis shows number of queries.

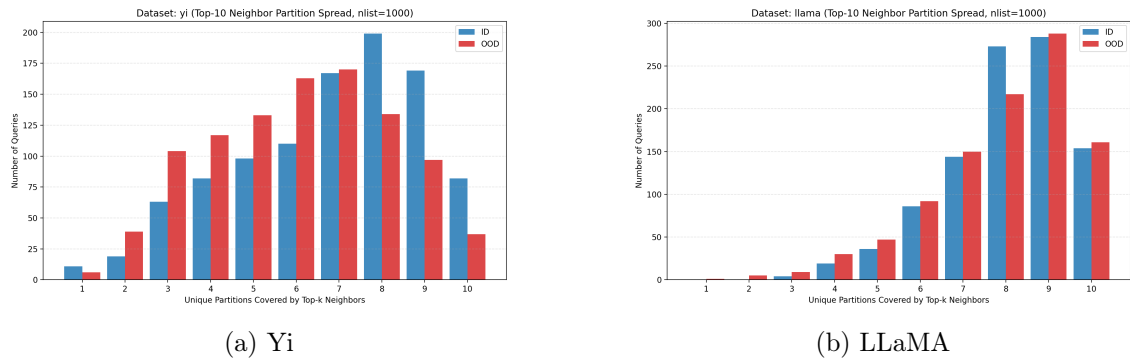


Figure 3.3: Distribution of neighbor scatteredness for $k = 10$ in sparse attention datasets. Blue bars correspond to ID queries, red bars to OOD queries. The x-axis shows neighbor scatteredness (1 to $k = 10$), and the y-axis shows number of queries.

3.2 Partition Probe Accuracy

Another observation is that, for queries exhibiting a high degree of out-of-distribution (OOD) behavior, the centroid-proximity-based partition probing strategy described in Section 1.2.2 becomes significantly less effective. In particular, it often fails to probe the partitions containing the nearest neighbors while selecting many redundant partitions that do not contain any of the top- k neighbors. Naturally, probing these unnecessary partitions is a primary contributor to the performance degradation observed in the previous chapter.

To quantify the effectiveness of partition probing, we introduce the notion of *Partition Probe Accuracy (PPA)*.

Definition (Partition Probe Accuracy). Let q be a query vector, $N_k(q)$ its set of k nearest neighbors, $P(x)$ the partition containing data vector x , and $\mathcal{P}_{n_{\text{probe}}}(q)$ the set of partitions probed for query q . The *Partition Probe Accuracy (PPA)* of q is defined as the fraction of probed partitions that contain at least one of the k nearest neighbors:

$$\text{PPA}(q, k, n_{\text{probe}}) = \frac{|\{P(x) \mid x \in N_k(q), P(x) \in \mathcal{P}_{n_{\text{probe}}}(q)\}|}{n_{\text{probe}}}.$$

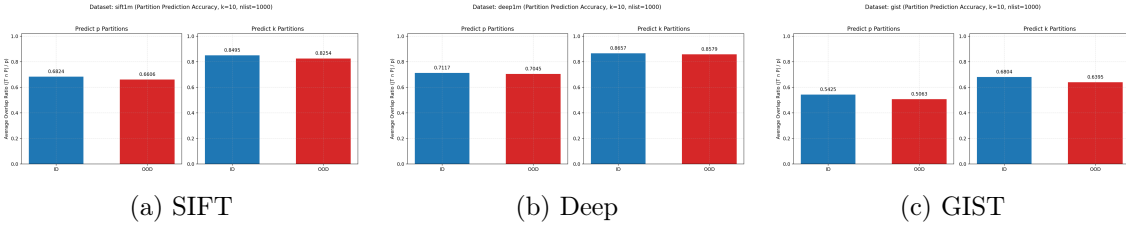


Figure 3.4: Partition probe accuracy for the SIFT, Deep, and GIST datasets. The left plot shows the probe accuracy when probing p partitions, where p is the actual number of partitions containing the top $k = 10$ nearest neighbors. The right plot shows the probe accuracy when probing a fixed $k = 10$ partitions. Blue bars correspond to the ID setting (data vectors used as queries), while red bars correspond to the OOD setting (the original query vectors provided with the datasets). The vertical axis represents the probe accuracy (%).

We evaluate Partition Probe Accuracy (PPA) under two settings. In the first setting, we compute $\text{PPA}(q, k, p)$, where p is the number of distinct partitions containing the k nearest neighbors of query q . This value is computed exhaustively for each query, providing the minimum number of partition probes required to achieve perfect recall (100%). In the second setting, we compute $\text{PPA}(q, k, k)$, where exactly k partitions are probed. This setting provides additional flexibility, since the k nearest neighbors can be distributed across at most k distinct partitions.

For the datasets whose queries exhibit a negligible degree of OOD behavior, there is no significant difference in Partition Probe Accuracy (PPA) between the ID setting, where data vectors are used as queries, and the OOD setting, where the original query vectors are used. As shown in Figure 3.4, the PPA values are nearly identical for both settings across the SIFT, Deep, and GIST datasets. These results are consistent with our previous observations regarding Mahalanobis distance, scatteredness, and recall-latency performance.

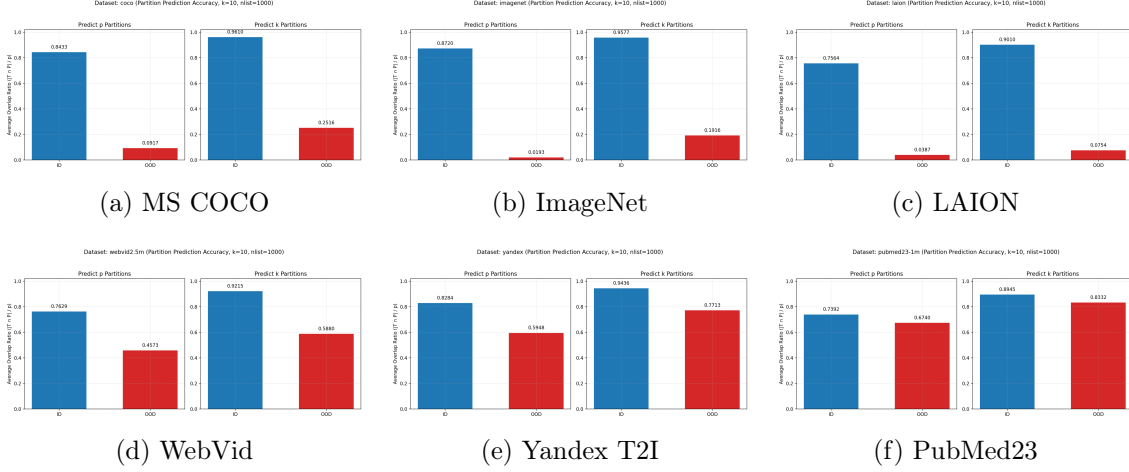


Figure 3.5: Partition probe accuracy for multi-modal/unimodal datasets. Left: probing p partitions (true number of partitions covering top $k = 10$ neighbors). Right: probing fixed $k = 10$ partitions. Blue = ID queries, red = OOD queries.

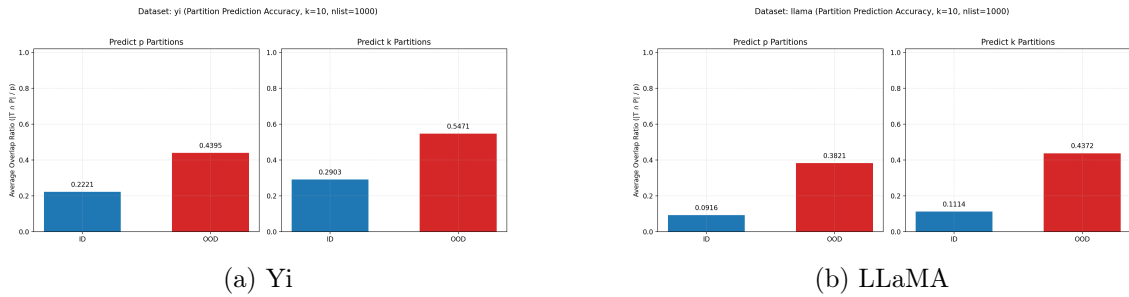


Figure 3.6: Partition probe accuracy for sparse attention datasets. Left: probing p partitions (true number of partitions covering top $k = 10$ neighbors). Right: probing fixed $k = 10$ partitions. Blue = ID queries, red = OOD queries.

For datasets exhibiting a high degree of OOD behavior, we observe a substantial gap in Partition Probe Accuracy (PPA) between the ID and OOD settings. As shown in Figure 3.5, for the MS COCO dataset, ID queries achieve a PPA of approximately 84%, whereas OOD queries achieve only about 9% when probing p partitions, where p is defined as above. When probing a fixed k partitions, the PPA increases to roughly 96% for ID queries but only 25% for OOD queries. This indicates that OOD queries may require probing approximately 4–9 times more partitions to achieve the same target recall, resulting in significantly higher query latency and corresponding performance degradation.

Similar trends are observed across the other datasets in this category. Moreover, the gap in PPA decreases as the Mahalanobis distance mean ratio becomes smaller. For example, Yandex T2I and PubMed23 exhibit only a 5–20% difference in PPA between the ID and OOD settings, which is consistent with our earlier observations.

For the sparse attention datasets, we again observe behavior that deviates from the expected trend, as the PPA of OOD queries is approximately 20–30% higher than that of ID queries, as shown in Figure 3.6. However, these results are consistent with our previous observations regarding performance (recall–latency) and scatteredness.

3.3 Distribution of Partition Probe Frequencies

The final observation regarding queries concerns how frequently each partition is probed. Specifically, for each $n_{\text{list}} = 1000$ partition, we count how often it is accessed when each query is allowed to probe $n_{\text{probe}} = 32$ partitions. Our observations reveal that as the degree of OOD behavior increases, the frequency distribution becomes increasingly skewed: a small set of partitions becomes very "hot", meaning they are probed by a large fraction of queries, while the remaining majority become "cold", i.e., they are rarely or never probed.

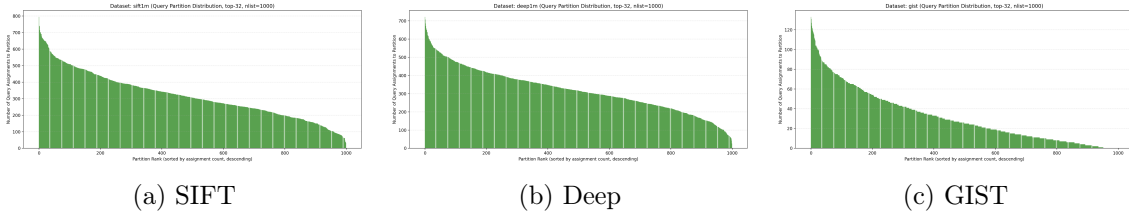


Figure 3.7: Distribution of partition probe frequencies for the SIFT, Deep, and GIST datasets using the original query vectors, with $n_{\text{list}} = 1000$ and $n_{\text{probe}} = 32$. The vertical axis shows the number of queries probing each partition, while the horizontal axis represents partition identifiers sorted in descending order of probe frequency.

For in-distribution (ID) datasets, the distribution is substantially less skewed, as illustrated in Figure 3.1. In contrast, datasets with strong out-of-distribution (OOD) behavior exhibit a highly skewed pattern. For instance, in the ImageNet dataset, only approximately 10% of partitions are probed at least once, while the remaining partitions are never visited, as shown in Figure 3.2.

Similar to retrieval datasets with high OOD behavior, the sparse attention datasets also exhibit a highly skewed distribution, although to a lesser extent, as illustrated in Figure 3.3.

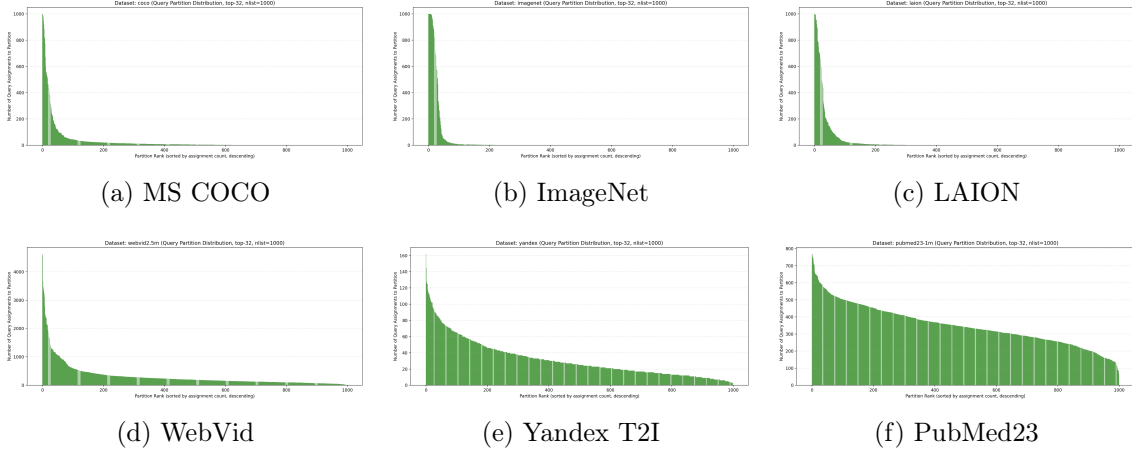


Figure 3.8: Distribution of partition probe frequencies for multi-modal/unimodal datasets using original query vectors, with $n_{\text{list}} = 1000$ and $n_{\text{probe}} = 32$. The y-axis shows number of queries per partition; the x-axis shows partitions sorted by decreasing frequency.

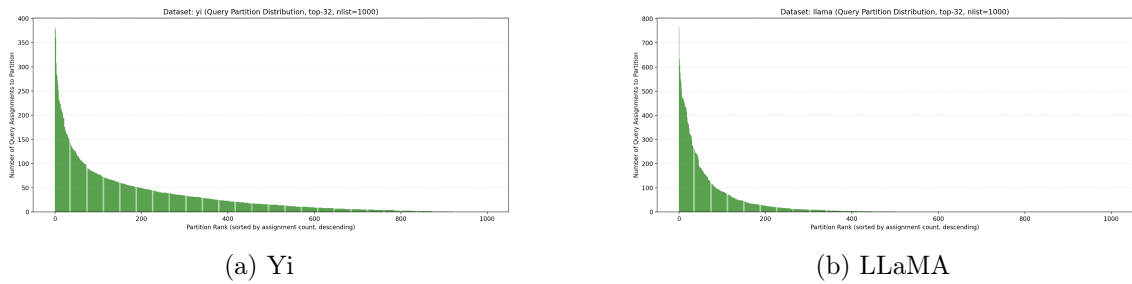


Figure 3.9: Distribution of partition probe frequencies for sparse attention datasets using original query vectors, with $n_{\text{list}} = 1000$ and $n_{\text{probe}} = 32$. The y-axis shows number of queries per partition; the x-axis shows partitions sorted by decreasing frequency.

Towards Out-of-Distribution Robust IVF Index Design

In the previous chapter, we investigated why traditional IVF indexes perform poorly under out-of-distribution (OOD) queries. In this chapter, we introduce several index designs that incorporate sample queries into the index construction process, making them more robust to OOD queries.

4.1 The Design Spectrum of Vector Search Indexes

Traditional vector search indexes do not take query vectors into account during index construction. Instead, they are built solely based on the data vectors, implicitly assuming that queries follow the same distribution as the indexed data. As demonstrated in the previous chapters, this assumption does not always hold, causing these indexes to perform poorly under out-of-distribution (OOD) queries.

Several recent studies have addressed this issue by incorporating a sample set of queries into the index construction process to improve robustness against OOD queries [2, 5, 13]. However, these approaches have been developed exclusively for graph-based indexes.

In this section, we explore the design space of IVF indexes with the goal of improving their robustness to out-of-distribution (OOD) queries.

Vector search indexes can be viewed as lying on a two-ended spectrum. So far, research on IVF indexes has focused almost exclusively on one end of this spectrum, where the index is constructed using only data vectors. At the opposite end lies an alternative design, in which the index is built solely from a sample set of query vectors.

4.1.1 Query-Driven IVF Index Construction: IVFFlat_Q

We explore the opposite end of this spectrum and refer to this indexing strategy as `IVFFlat_Q`. The idea is straightforward: assuming access to a representative sample of query vectors, we perform the clustering algorithm (K-means) on the query vectors instead of the data vectors to learn the partition centroids. The data vectors are then assigned to these partitions based on their proximity to the learned centroids, as in the standard IVF construction. Query processing remains unchanged: given a query q , the nearest centroids are identified, and the corresponding partitions are probed.

4.1.2 A Hybrid IVF Index Construction Strategy: QueryWeighted_IVFFlat

We propose a hybrid index construction strategy, called **QueryWeighted_IVFFlat**, which combines information from both the data distribution and a sample of query-induced neighborhood structure. The main idea is to bias centroid learning toward data vectors that are frequently retrieved by sample queries, thereby emphasizing regions of the space that are more relevant at query time.

Let $\mathcal{X} = \{x_i\}_{i=1}^N$ denote the dataset and $\mathcal{Q} = \{q_j\}_{j=1}^M$ a set of sample queries. We first compute the exact k -nearest neighbors of each query q_j over $\mathcal{X}$, and define a frequency score for each data vector x_i as:

$$w_i = |\{(q_j, x_i) : x_i \in N_k(q_j), q_j \in \mathcal{Q}\}|,$$

where $N_k(q_j)$ denotes the k nearest neighbors of query q_j .

These weights induce a weighted dataset for centroid learning. Specifically, we construct an augmented multiset $\tilde{\mathcal{X}}$ in which each data vector x_i is repeated w_i times:

$$\tilde{\mathcal{X}} = \bigcup_{i=1}^N \{x_i \text{ repeated } w_i \text{ times}\}.$$

We then perform K-means clustering on $\tilde{\mathcal{X}}$ to obtain n_{list} centroids. After centroid computation, each original data vector $x_i \in \mathcal{X}$ is assigned to its nearest centroid in the standard manner, without duplication.

Query processing remains identical to standard IVF: given a query q , we identify the closest centroids and probe the corresponding partitions.

This construction yields a hybrid index that incorporates query-induced importance into the partitioning process while preserving the standard IVF query-time procedure.

4.2 Impact of Query Distribution Information Injection into Index Construction on Performance

We now compare IVFFlat with the two proposed indexing strategies which use a sample set of queries.

We constructed all indexes using $n_{\text{list}} = 1000$ partitions. For the construction of **IVFFlat_Q** and **QueryWeighted_IVFFlat**, we split the available queries into two equal halves: one half was used during index construction, while the remaining half was reserved for evaluation.

As shown in Figure 4.1, there is no noticeable performance difference among the three indexing strategies on the in-distribution (ID) datasets, as expected. Since the query and data vectors are drawn from the same underlying distribution, constructing the index using only query vectors in **IVFFlat_Q** produces partitions that are nearly identical to those obtained by the standard **IVFFlat**. Likewise, the additional weighting employed by **QueryWeighted_IVFFlat** provides little benefit, as there is no skew in the regions of the data distribution accessed by the queries.

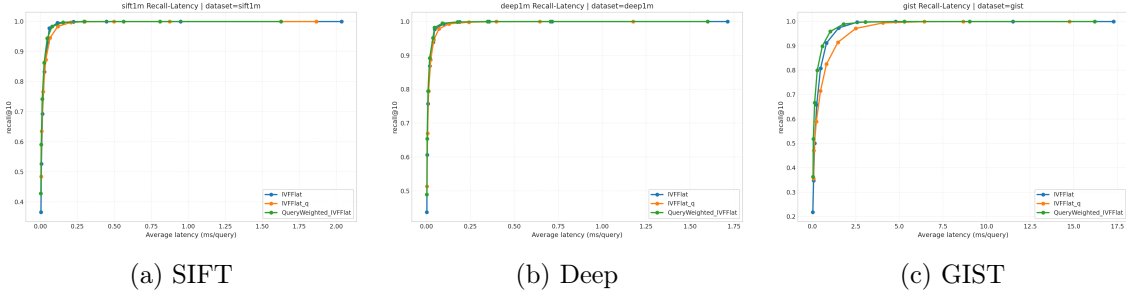


Figure 4.1: Recall–latency curves for the in-distribution (ID) datasets using the IVFFlat, IVFFlat_Q, and QueryWeighted_IVFFlat indexes with $n_{\text{list}} = 1000$ partitions. The vertical axis represents Recall@10, while the horizontal axis shows query latency (ms). The blue, orange, and green curves correspond to IVFFlat, IVFFlat_Q, and QueryWeighted_IVFFlat, respectively.

For the out-of-distribution (OOD) datasets, incorporating query distribution information into the index construction process leads to substantial performance improvements, as evidenced by the recall–latency curves of QueryWeighted_IVFFlat and IVFFlat_Q in Figure 4.2. While QueryWeighted_IVFFlat consistently outperforms the standard IVFFlat across all datasets, IVFFlat_Q performs worse than the baseline on the MS COCO dataset. This suggests that relying exclusively on the query distribution, as in IVFFlat_Q, can overfit the index to the sampled queries and may degrade performance in certain cases.

We also observe that the performance gains obtained by incorporating query distribution information are substantially larger for datasets such as ImageNet, while they remain relatively limited for datasets such as Yandex T2I. This observation is consistent with our earlier findings based on the Mahalanobis distance, which indicate that datasets exhibiting a higher degree of OOD behavior benefit more from query-aware index construction.

Similar to the MS COCO dataset, IVFFlat_Q fails to outperform the baseline IVFFlat on the sparse attention datasets, as shown by the recall–latency curves in Figure 4.2. This is because IVFFlat_Q does not partition the data vectors evenly. In particular, a small number of partitions end up containing the vast majority of the data vectors, while many other partitions remain nearly empty, effectively reducing the available partitioning capacity of the index. In contrast, QueryWeighted_IVFFlat consistently achieves significantly better performance than the baseline. These results suggest that an effective index for OOD datasets should leverage information from both the query and data distributions, rather than relying exclusively on either one.

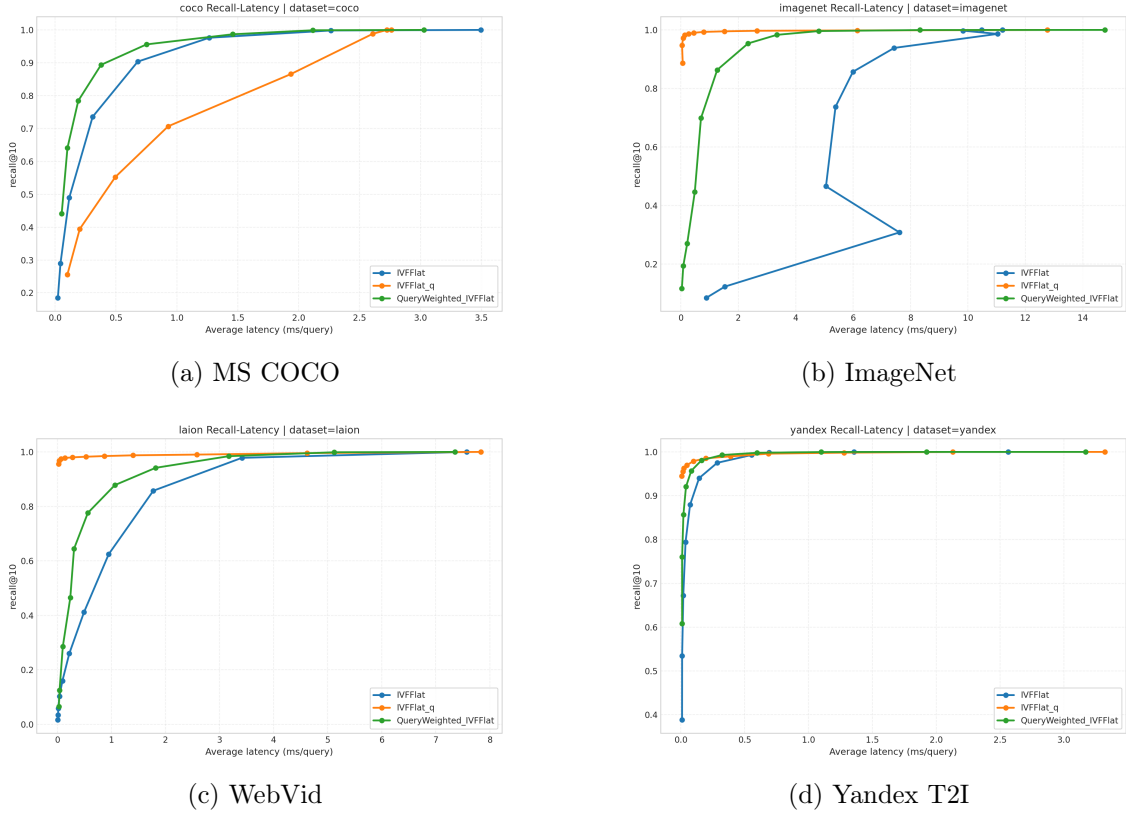


Figure 4.2: Recall-latency curves for the out-of-distribution (OOD) retrieval datasets using the IVFFlat, IVFFlat_Q, and QueryWeighted_IVFFlat indexes with $n_{\text{list}} = 1000$ partitions. The vertical axis represents Recall@10, while the horizontal axis shows query latency (ms). The blue, orange, and green curves correspond to IVFFlat, IVFFlat_Q, and QueryWeighted_IVFFlat, respectively.

4.3 Understanding the Performance Gains of Query-Aware Index Construction

In the previous chapter, we observed that IVFFlat, our baseline index that is unaware of the query distribution, suffers from performance degradation on out-of-distribution (OOD) datasets. This degradation is primarily caused by higher scatteredness and lower Partition Probe Accuracy (PPA). In this chapter, we analyze how the proposed query-distribution-aware indexing strategies affect these metrics and explain how these changes translate into performance improvements.

For OOD retrieval datasets such as ImageNet, we observe that the scatteredness of nearest neighbors across partitions is significantly reduced for the IVFFlat_Q index. As shown in Figure 4.4, IVFFlat_Q requires only 1.31 partitions on average to retrieve the $k = 10$ nearest neighbors, whereas IVFFlat and QueryWeighted_IVFFlat require 4.66 and 3.68 partitions on average, respectively. This reduction in scatteredness directly contributes to the improved recall-latency performance of IVFFlat_Q, as observed in the previous experiments.

Another major factor contributing to performance degradation is low Partition Probe Accuracy (PPA). As shown in Figure 4.5, IVFFlat_Q achieves a significantly higher PPA of

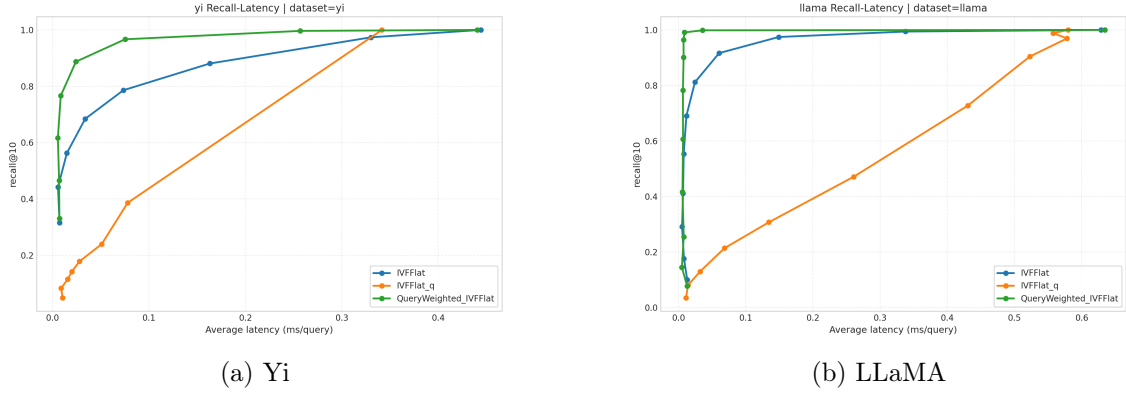


Figure 4.3: Recall-latency curves for the out-of-distribution (OOD) sparse attention datasets using the IVFFlat, IVFFlat_Q, and QueryWeighted_IVFFlat indexes with $n_{\text{list}} = 1000$ partitions. The vertical axis represents Recall@10, while the horizontal axis shows query latency (ms). The blue, orange, and green curves correspond to IVFFlat, IVFFlat_Q, and QueryWeighted_IVFFlat, respectively.

around 90%, whereas IVFFlat and QueryWeighted_IVFFlat achieve only approximately 5% and 33%, respectively. This substantial improvement in PPA indicates that IVFFlat_Q is much more effective at identifying relevant partitions, which is the primary reason behind its superior recall-latency performance.

Unlike OOD retrieval datasets, we observe that IVFFlat_Q does not achieve high Partition Probe Accuracy (PPA) on sparse attention datasets. For the LLaMA dataset, its PPA is only around 12%, while QueryWeighted_IVFFlat achieves approximately 45%, which is only a moderate improvement over the baseline IVFFlat performance of around 33%, as shown in Figure 4.7. Regarding scatteredness, QueryWeighted_IVFFlat exhibits significantly higher scatteredness for the LLaMA dataset, as shown in Figure 4.6. These observations explain why IVFFlat_Q performs worse than the baseline and why the improvement of QueryWeighted_IVFFlat remains limited for LLaMA and, more generally, sparse attention datasets.

Overall, these results suggest that an ideal OOD-resilient index should incorporate information from both the query and data distributions rather than relying exclusively on either one. Although QueryWeighted_IVFFlat is not a complete solution, as indicated by its remaining high scatteredness on LLaMA, it provides an important step toward designing more robust IVF indexes for OOD queries.

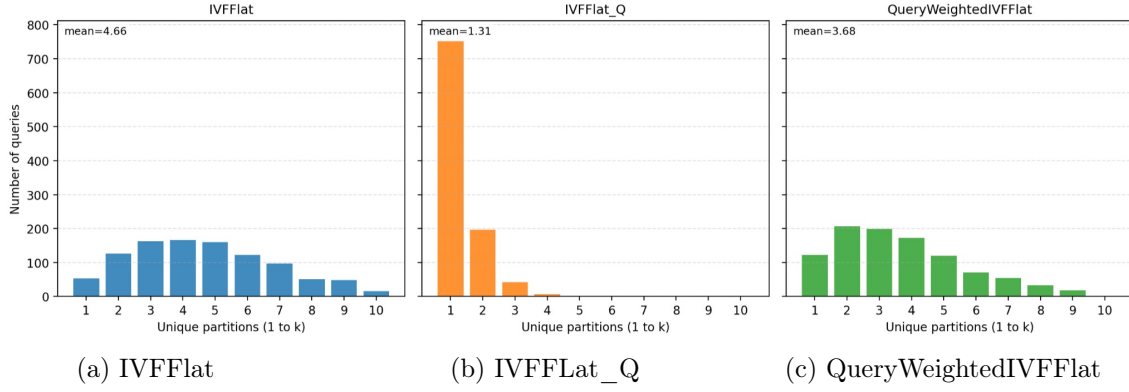


Figure 4.4: Scatteredness distributions for the IVFFlat, IVFFlat_Q, and QueryWeighted_IVFFlat indexes with $n_{\text{list}} = 1000$ partitions on the ImageNet dataset.

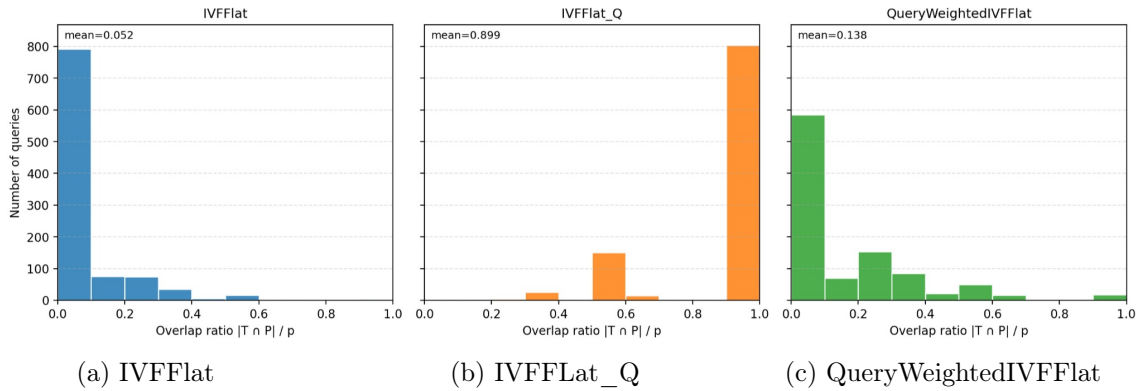


Figure 4.5: Distribution of Partition Probe Accuracy (PPA) for the IVFFlat, IVFFlat_Q, and QueryWeighted_IVFFlat indexes with $n_{\text{list}} = 1000$ partitions on the ImageNet dataset.

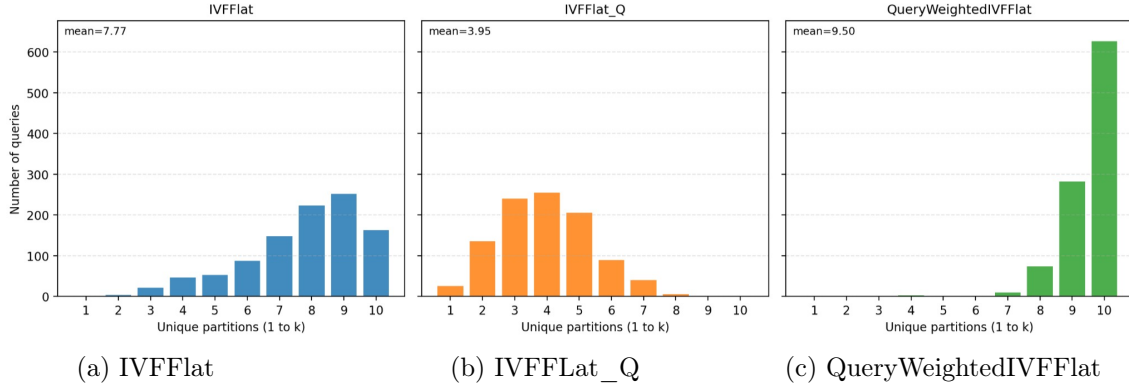


Figure 4.6: Scatteredness distributions for the IVFFlat, IVFFlat_Q, and QueryWeighted_IVFFlat indexes with $n_{\text{list}} = 1000$ partitions on the LLaMA dataset.

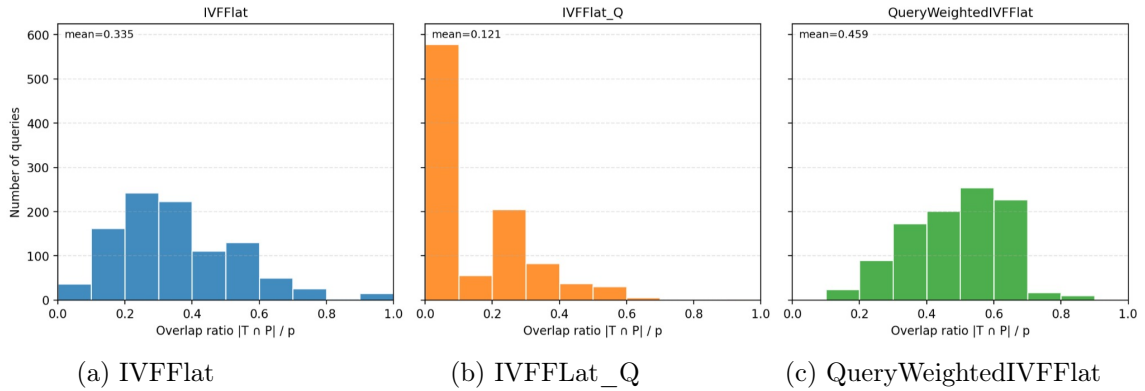


Figure 4.7: Distribution of Partition Probe Accuracy (PPA) for the IVFFlat, IVFFlat_Q, and QueryWeighted_IVFFlat indexes with $n_{\text{list}} = 1000$ partitions on the LLaMA dataset.

Bibliography

- [1] Manos Chatzakis, Yannis Papakonstantinou, and Themis Palpanas. “DARTH: Declarative Recall Through Early Termination for Approximate Nearest Neighbor Search”. In: *Proceedings of the ACM on Management of Data* 3.4 (2025), pp. 1–26. ISSN: 2836-6573. DOI: [10.1145/3749160](https://doi.org/10.1145/3749160). URL: <http://dx.doi.org/10.1145/3749160>.
- [2] Meng Chen et al. “RoarGraph: A Projected Bipartite Graph for Efficient Cross-Modal Approximate Nearest Neighbor Search”. In: *Proceedings of the VLDB Endowment* 17.11 (2024), pp. 2735–2749. ISSN: 2150-8097. DOI: [10.14778/3681954.3681959](https://doi.org/10.14778/3681954.3681959). URL: <http://dx.doi.org/10.14778/3681954.3681959>.
- [3] Jacob Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019. arXiv: [1810.04805](https://arxiv.org/abs/1810.04805) [cs.CL]. URL: <https://arxiv.org/abs/1810.04805>.
- [4] Elias Jääsaari et al. *VIBE: Vector Index Benchmark for Embeddings*. 2025. arXiv: [2505.17810](https://arxiv.org/abs/2505.17810) [cs.LG]. URL: <https://arxiv.org/abs/2505.17810>.
- [5] Shikhar Jaiswal et al. *OOD-DiskANN: Efficient and Scalable Graph ANNS for Out-of-Distribution Queries*. 2022. arXiv: [2211.12850](https://arxiv.org/abs/2211.12850) [cs.LG]. URL: <https://arxiv.org/abs/2211.12850>.
- [6] Hervé Jégou et al. *Searching in one billion vectors: re-rank with source coding*. 2011. arXiv: [1102.3828](https://arxiv.org/abs/1102.3828) [cs.IR]. URL: <https://arxiv.org/abs/1102.3828>.
- [7] Yu. A. Malkov and D. A. Yashunin. *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*. 2018. arXiv: [1603.09320](https://arxiv.org/abs/1603.09320) [cs.DS]. URL: <https://arxiv.org/abs/1603.09320>.
- [8] Aude Oliva and Antonio Torralba. “Modeling the Shape of the Scene: A Holistic Representation of the Spatial Envelope”. In: *International Journal of Computer Vision* 42.3 (May 2001), pp. 145–175. ISSN: 1573-1405. DOI: [10.1023/A:1011139631724](https://doi.org/10.1023/A:1011139631724). URL: <https://doi.org/10.1023/A:1011139631724>.
- [9] Alec Radford et al. *Learning Transferable Visual Models From Natural Language Supervision*. 2021. arXiv: [2103.00020](https://arxiv.org/abs/2103.00020) [cs.CV]. URL: <https://arxiv.org/abs/2103.00020>.
- [10] Suhas Jayaram Subramanya et al. “DiskANN: fast accurate billion-point nearest neighbor search on a single node”. In: *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Red Hook, NY, USA: Curran Associates Inc., 2019.
- [11] Eric S. Tellez et al. “Overview of the SISAP 2025 Indexing Challenge”. In: *Similarity Search and Applications: 18th International Conference, SISAP 2025, Reykjavik, Iceland, October 1–3, 2025, Proceedings*. Reykjavik, Iceland: Springer-Verlag, 2025, pp. 403–414. ISBN: 978-3-032-06068-6. DOI: [10.1007/978-3-032-06069-3_33](https://doi.org/10.1007/978-3-032-06069-3_33). URL: https://doi.org/10.1007/978-3-032-06069-3_33.
- [12] Artem Babenko Yandex and Victor Lempitsky. “Efficient Indexing of Billion-Scale Datasets of Deep Descriptors”. In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 2055–2063. DOI: [10.1109/CVPR.2016.226](https://doi.org/10.1109/CVPR.2016.226).

- [13] Qiang Yue et al. “Efficient and Robust Out-Of-Distribution Vector Similarity Search with Cross-Distribution Monotonic Graph”. In: *Proc. ACM Manag. Data* 4.1 (Apr. 2026). DOI: [10.1145/3786643](https://doi.org/10.1145/3786643). URL: <https://doi.org/10.1145/3786643>.